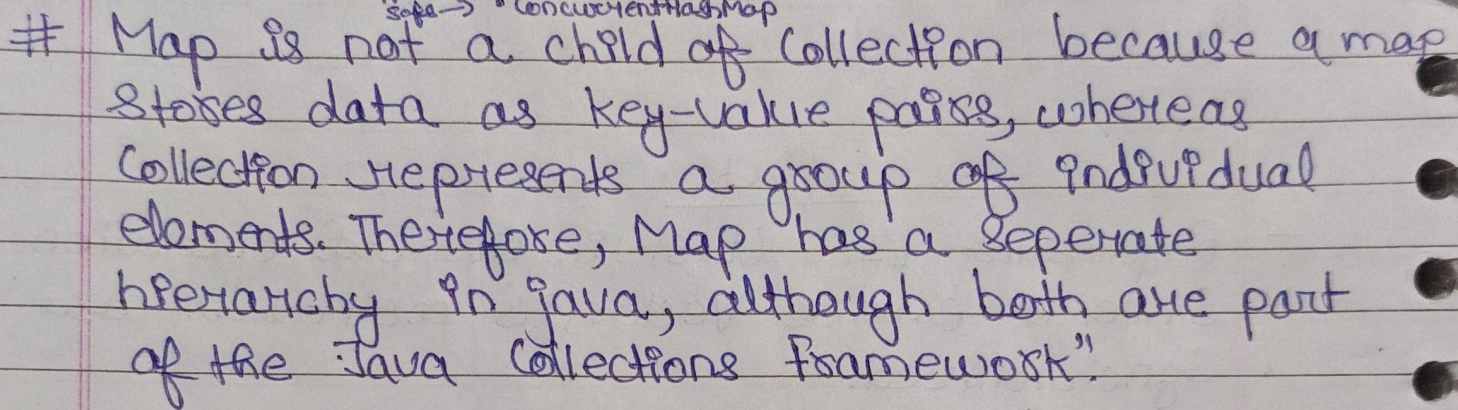


Page No :-
Date :- / /



Map Properties:

- It's an interface and its implementations are:
 - HashMap: do not maintain the order
 - Hashtable: Synchronized version of HashMap
 - LinkedHashMap: Maintains the insertion order
 - TreeMap: Sorts the data internally.
- Object that maps key to values.
- Can not contain duplicate key.

Methods available in Map Interface

SrNo.	Method Name	Usage
1.	size()	returns the no. of key-value mapping present.
2.	isEmpty()	true: if map contains no-key-value mapping else return false.
3.	containsKey(Object key)	- if given key is already present in the map return true else false.
4.	containsValue(Object o)	- return true, if one or more key mapped to the specified value.
5.	get(Object key)	- return the value to which this key is mapped.
6.	put(K key, V value)	- if map already has the same key present $\rightarrow$ it will overwrite the value else do not have the key present $\rightarrow$ it will add new key-value mapping.
7.	remove(Object key)	- removes the key value mapping from the map for the specified key.
8.	putAll(Map<K, V> m)	- insert the mappings from the specified map to this map.
9.	clear()	- removes all the mappings from the map and make it empty.
10.	Set<K> keySet()	- returns the set view of the key contained in the Map. Set is backed by the Map, so changes in the map, will be reflected in the set and vice-versa.
11.	Collection<V> values()	- returns the collection view of all the values.
12.	Set<Map.Entry<K, V>> entrySet()	- returns the set view of the mappings present in the Map.

- | | | |
|-----|--|---|
| 13. | <code>putIfAbsent(K key, V value)</code> | - If key already exist return the value already associated, else create a new mapping with this key and value |
| 14. | <code>getOrDefault(key, defaultvalue)</code> | - If key does not exist or value is null, it returns the default value |
| 15. | Entry sub-interface | - Entry is the sub-Interface of Map. So, its accessed via <code>Map.Entry</code> |

LinkedHashMap

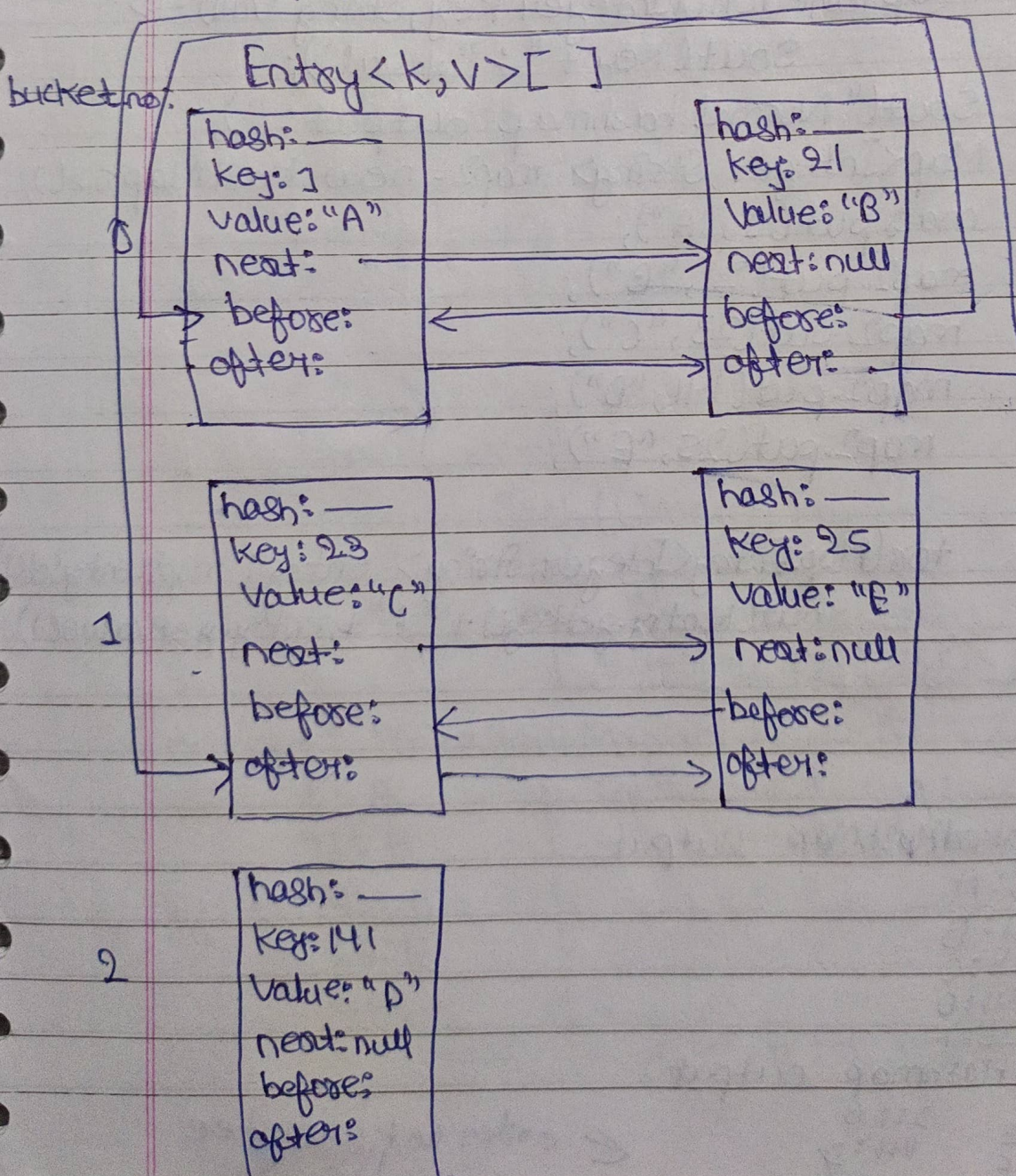
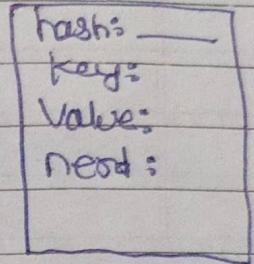
LinkedHashMap:

①

1. Helps in maintain insertion order or helps in maintain Access^② Order. (less freq used to highly freq used)
2. Similary to Hashmap, but also uses Double LinkedList

Ex: map.put(1, "A");
map.put(21, "B");
map.put(23, "C");
map.put(141, "D");
map.put(25, "E");

Normal HashMap Entry node Structure is:



```
public class LinkedHashMapExample {
    public static void main(String[] args) {
        System.out.println("LinkedHashMap Output");
        Map<Integer, String> map = new LinkedHashMap<>();
        map.put(1, "A");
        map.put(21, "B");
        map.put(23, "C");
        map.put(141, "D");
        map.put(25, "E");
```

```
        map.forEach((Integer key, String val) ->
            System.out.println(key + ":" + val));
```

```
        System.out.println("Normal hashmap output:");
        Map<Integer, String> map2 = new HashMap<>();
        map2.put(1, "A");
        map2.put(21, "B");
        map2.put(23, "C");
        map2.put(141, "D");
        map2.put(25, "E");
```

```
        for (Map.Entry<Integer, String> entry : map2.entrySet()) {
            System.out.println(entry.getKey() + ":" + entry.getValue());
        }
    }
}
```

Output: LinkedHashMap Output

1:A
21:B
23:C
141:D
25:E

Normal Hashmap output

1:A 21:B
25:E 141:D
23:C

← order not guarantee